

Virtual Engineering Teams as a Service

AI-Powered Dev Teams, Human-Led Architecture

Business Plan

Prepared for Sean Rohde

June 26, 2026 | Version 1.0 | Confidential

Virtual Engineering Teams as a Service (VETaaS)

Business Plan -- "AI-Powered Dev Teams, Human-Led Architecture"

Prepared for: Sean Rohde, Senior AI Engineer

Date: June 26, 2026

Version: 1.0 -- Draft for Discussion

1. Executive Summary

Small and mid-sized companies need custom software but face a brutal reality: hiring even one developer costs \$80-120K/year fully loaded, freelancers are unreliable, and agencies charge \$150-250/hour. Most projects never get built.

VETaaS solves this by giving non-technical companies a dedicated 3-agent AI engineering team overseen by a Senior AI Engineer (Sean), for \$1,500-2,500/month -- roughly 25% of what a single junior developer costs.

The key differentiator: clients never touch the agents. They describe what they want. They see results every week -- kanban boards, bug trackers, deployed software on test servers, and production deployments. Sean translates their needs into architecture, reviews every line before it ships, and owns quality. The AI does the typing; Sean does the thinking.

2. The Product: What Clients Actually Get

2.1 The Deliverables (Monthly Subscription)

Deliverable	Description
3 Dedicated AI Agents	Backend, Frontend, and DevOps -- each specialized, each working on the client's codebase
Senior AI Engineer Oversight	Sean architects the system, writes tickets, reviews all agent output, and ensures quality
Client Portal Access	Real-time dashboard showing progress, kanban boards, bug status, and deployment history
QA Test Server	Deploy preview environment where the client's team can test before production
Production Server	Live deployed application (staging -> production after client approval)
Weekly Progress Report	Auto-generated summary: what shipped, what's in progress, what's blocked, what's next
Bug Tracker	Client-submitted bugs visible with status (reported -> in progress -> fixed -> deployed)
Source Code Ownership	Client owns the code. Full git history. No lock-in -- cancel anytime and take everything
Communication Channel	Email/Slack/Teams -- Sean is reachable. No prompting, no technical jargon required

2.2 What Clients Do NOT Get (And Why That's Good)

They Don't Get	Why
Direct access to agents	Agents are powerful but need architectural direction. Bad prompts = bad code. Sean is the quality gate.
Unlimited feature requests	Scope is managed through prioritized kanban. Prevents scope creep and ensures steady progress.
Real-time chat with agents	Clients describe needs in plain language. Sean translates to tickets. No technical knowledge required.
Ability to modify code directly	All changes go through review. Prevents "I tried to fix it and broke everything."

3. System Architecture



```

graph TD
    subgraph Client_Box [CLIENT]
        direction TB
        C_Portal[PORTAL]
        C_Kanban[Kanban]
        C_Board[Board]
        C_Bugs[Bugs]
        C_Next[Next]
        C_Deploy[Deploy]
        C_Log[Log]
    end

    subgraph Weekly_Report [WEEKLY REPORT (email)]
        direction TB
        WR_Shipped[- Shipped]
        WR_InProg[- In Prog]
        WR_Bugs[- Bugs]
        WR_Blocked[- Blocked]
        WR_Next[- Next]
    end

    subgraph QA_Test [QA TEST SERVER]
        direction TB
        QT_Staging[Staging]
        QT_Deploy[deploy]
        QT_ClientQA[Client QA]
        QT_Tests[tests here]
    end

    subgraph Production_Server [PRODUCTION SERVER]
        direction TB
        PS_LiveApp[Live app]
        PS_Production[production]
        PS_URL[URL]
    end

    C_Portal --> WR_Shipped
    C_Portal --> WR_InProg
    C_Portal --> WR_Bugs
    C_Portal --> WR_Blocked
    C_Portal --> WR_Next
    C_Portal --> QT_Staging
    C_Portal --> QT_Deploy
    C_Portal --> PS_LiveApp
    C_Portal --> PS_Production
    C_Portal --> PS_URL

    WR_Shipped --> QA_Test
    WR_InProg --> QA_Test
    WR_Bugs --> QA_Test
    WR_Blocked --> QA_Test
    WR_Next --> QA_Test

    QA_Test --> Production_Server

    Production_Server --> SEAN[SEAN'S ORCHESTRATION LAYER]

    SEAN --> AGENT_ALPHA[AGENT ALPHA]
    SEAN --> AGENT_BETA[AGENT BETA]
    SEAN --> AGENT_GAMMA[AGENT GAMMA]

    AGENT_ALPHA --> C_Kanban
    AGENT_ALPHA --> C_Board
    AGENT_ALPHA --> C_Bugs
    AGENT_ALPHA --> C_Next
    AGENT_ALPHA --> C_Deploy
    AGENT_ALPHA --> C_Log

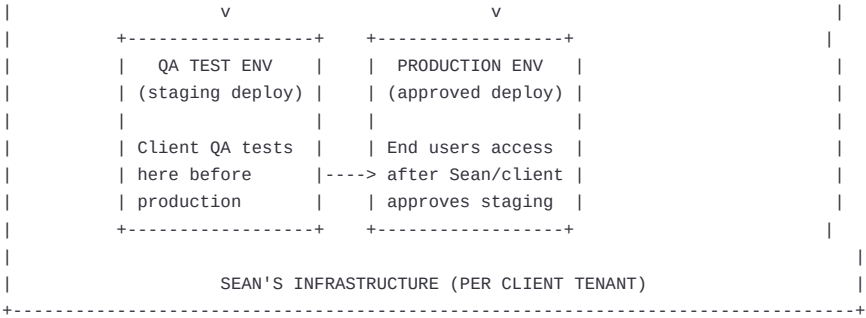
    AGENT_BETA --> C_Kanban
    AGENT_BETA --> C_Board
    AGENT_BETA --> C_Bugs
    AGENT_BETA --> C_Next
    AGENT_BETA --> C_Deploy
    AGENT_BETA --> C_Log

    AGENT_GAMMA --> C_Kanban
    AGENT_GAMMA --> C_Board
    AGENT_GAMMA --> C_Bugs
    AGENT_GAMMA --> C_Next
    AGENT_GAMMA --> C_Deploy
    AGENT_GAMMA --> C_Log

    AGENT_ALPHA --> AGENT_BETA
    AGENT_BETA --> AGENT_GAMMA
    AGENT_GAMMA --> AGENT_ALPHA

    AGENT_ALPHA --> SEAN
    AGENT_BETA --> SEAN
    AGENT_GAMMA --> SEAN
  
```

The diagram illustrates the 'SEAN'S ORCHESTRATION LAYER' architecture. At the top, a 'CLIENT' box contains a 'PORTAL' and a 'WEEKLY REPORT (email)' section. The report lists items: '- Shipped', '- In Prog', '- Bugs', '- Blocked', and '- Next'. Below the report is a 'QA TEST SERVER' box with 'Staging', 'deploy', 'Client QA', and 'tests here'. To the right is a 'PRODUCTION SERVER' box with 'Live app', 'production', and 'URL'. Arrows show data flow from the Client Portal to the Weekly Report, then to the QA Test Server, and finally to the Production Server. Below the servers is the 'SEAN'S ORCHESTRATION LAYER' box, which contains three AI agents: 'AGENT ALPHA' (Backend Engineer), 'AGENT BETA' (Frontend Engineer), and 'AGENT GAMMA' (DevOps Engineer). Each agent has a list of responsibilities: Alpha handles API development, database design, business logic, auth/security, and testing; Beta handles React/Next.js, UI components, responsive CSS, state mgmt, and accessibility; Gamma handles CI/CD pipelines, Docker/K8s, cloud infra, deployments, and monitoring. Arrows show bidirectional communication between the agents and between each agent and the SEAN layer. At the bottom, a 'SHARED CLIENT REPO (GitHub / GitLab)' is shown, with arrows indicating that all three agents interact with it for feature branches, PRs, merges, and CI/CD auto-deploys.



3.1 Infrastructure Per Client

Component	Provider	Monthly Cost (est.)
Agent Alpha VPS (Backend)	Hetzner / DigitalOcean	\$12-24
Agent Beta VPS (Frontend)	Hetzner / DigitalOcean	\$12-24
Agent Gamma VPS (DevOps)	Hetzner / DigitalOcean	\$12-24
Test Environment Server	Railway / Fly.io / VPS	\$10-20
Production Server	Railway / Fly.io / VPS	\$10-50
API Usage (model inference)	DeepSeek / Anthropic / OpenAI	\$100-300
GitHub Private Repo	GitHub	\$0-4
Client Portal Hosting	Vercel / Netlify	\$0-20
Domain / DNS	Cloudflare	\$0-10
Total per client		**~\$156-476/month**

4. Pricing Tiers

4.1 Tier Structure

	SOLO AGENT	SMALL TEAM	GROWTH TEAM
	-----	-----	-----
Agents	1 agent	3 agents	5 agents
Monthly Price	\$1,000/mo	\$2,000/mo	\$3,500/mo
Best For	Tiny projects, single feature, bug fixes	Most businesses, web apps, SaaS MVP, internal tools	Larger scope, platform builds, complex systems
Includes:			

Senior oversight	[x]	[x]	[x]
Client portal	[x]	[x]	[x]
QA test server	[x]	[x]	[x]
Production server	[x]	[x]	[x]
Weekly reports	[x]	[x]	[x]
Bug tracker	[x]	[x]	[x]
Code ownership	[x]	[x]	[x]
Priority support	--	[x]	[x]
Architecture docs	--	[x]	[x]
Dedicated QA agent	--	--	[x]
CI/CD pipeline	[x]	[x]	[x]
Uptime monitoring	--	[x]	[x]

4.2 Enterprise / Custom

For clients with compliance needs (HIPAA, SOC2), dedicated infrastructure, or >5 agents: custom quote starting at \$5,000/month.

4.3 The Math That Makes It Work

	Per Client (Small Team)
Revenue	\$2,000/month
Hard Costs (servers + API)	~\$300/month
Gross Margin	**85%**
Sean's Time	~5-8 hours/month
Sean's Effective Rate	~\$213-340/hour

At 10 Small Team clients:

- Monthly Revenue: \$20,000
- Monthly Costs: ~\$3,000
- Sean's Time: ~50-80 hours/month (manageable with processes)
- Monthly Gross Profit: ~\$17,000

5. The Client Portal

5.1 What It Looks Like (Client-Facing)

```

+-----+
|  VETaaS Client Portal                [Client: AcmeCorp]  |
+-----+
|
|  +-----+ +-----+ +-----+ +-----+ |
|  | PROGRESS | | TICKETS | |  BUGS  | |  SERVERS | |
|  | DASHBOARD | | (Kanban) | |  TRACKER | |  STATUS | |
|  |         | |         | |         | |         | |
|  | #####... | | To Do:  4 | | Open:   2 | | QA:  [P2] | |
|  | 75% done  | | Doing:  2 | | In Prog: 1 | | Prod: [P2] | |
|  |         | | Review:  1 | | Fixed:  5 | |         | |
|  | 3 features | | Done:   12 | |         | | [[Open QA] | |
|  | shipped    | |         | |         | | [[Open Prod]| |
|  | this month | | [View All] | | [Report Bug]| |         | |
|  +-----+ +-----+ +-----+ +-----+ |
|
|  +-----+
|  | THIS WEEK'S REPORT                June 20-26  |
|  +-----+
|  |
|  | [OK] SHIPPED:
|  |   - User authentication with Google OAuth
|  |   - Dashboard analytics page with 6 chart types
|  |   - Fixed checkout bug (incorrect tax calculation)
|  |
|  | [~] IN PROGRESS:
|  |   - Payment integration with Stripe (est. June 30)
|  |   - Mobile responsive navigation (est. June 28)
|  |
|  | [X] BLOCKED:
|  |   - Email notifications -- waiting on SendGrid API key
|  |     from client
|  |
|  | [*] NEXT WEEK:
|  |   - Stripe checkout flow completion
|  |   - Admin user management panel
|  |   - Database backup automation
|  |
|  | [>] TEST ENVIRONMENT: https://qa.acmecorp.vetaas.io
|  | [>] PRODUCTION:      https://app.acmecorp.com
|  |
|  +-----+
|

```

+-----+

5.2 Portal Features Breakdown

Module	What Client Sees	What Client Does
Progress Dashboard	% complete, features shipped, velocity chart	View only -- progress is reported, not managed by client
Kanban Board	Columns: Backlog -> In Progress -> Review -> Done	View only -- Sean prioritizes and assigns
Bug Tracker	Submitted bugs, status, resolution notes	**Submit bugs** (only write action available)
Deployment Log	Date, version, what changed, who approved	View only + link to test/prod
Weekly Reports	Auto-generated summary archive	View + download PDF
Server Status	Green/red indicators for test and prod	Clickable links to deployed environments
Feature Requests	Simple form: "I need X so that Y"	**Submit requests** (second write action)

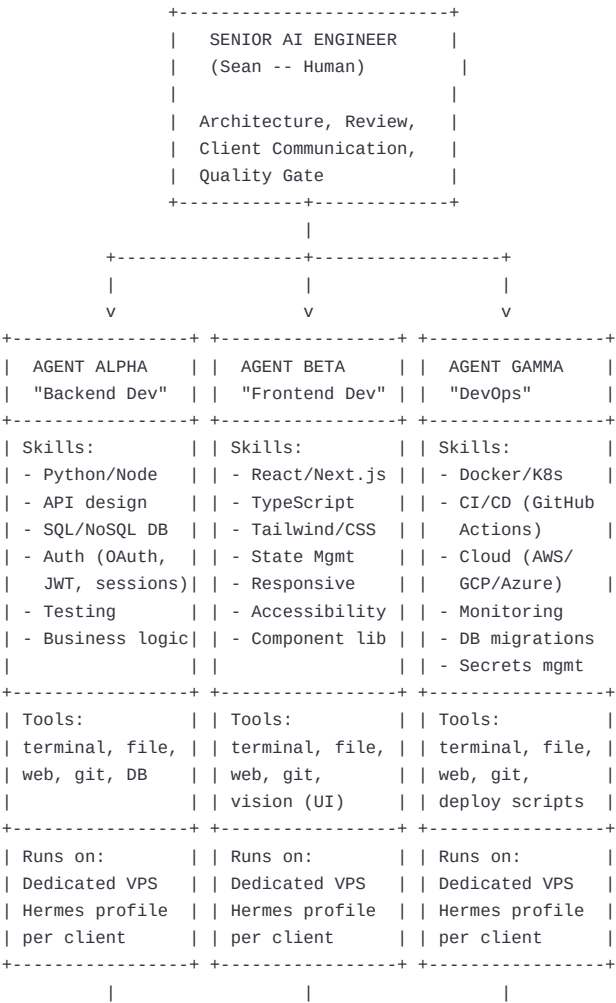
5.3 The Only Two Write Actions Clients Get

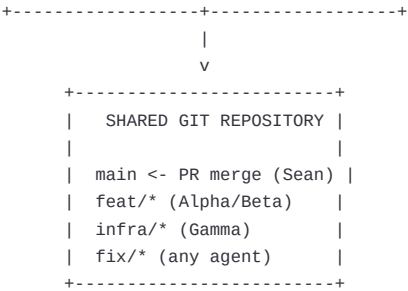
1. Submit a Bug -- Title, description, steps to reproduce, screenshot upload
2. Submit a Feature Request -- Plain language: "I want customers to be able to..." -- no technical details required

Everything else is read-only. They see the machine running. They don't touch the controls.

6. The Agent Team Structure

6.1 Agent Roles & Specialization





6.2 How Agents Collaborate

- 1. Sean creates tickets in the repo (GitHub Issues or a project board)
- 2. Each agent works its lane:
 - Alpha builds the API endpoint
 - Beta builds the UI that consumes it
 - Gamma handles deployment, environment config, and monitoring
- 3. Agents open PRs against their feature branches
- 4. Sean reviews and merges -- this is the quality gate
- 5. Gamma deploys merged code to staging automatically
- 6. Client QA tests on staging
- 7. Sean approves production deploy after QA passes

7. The Workflow: From Client Request to Deployed Feature

WEEK 1		WEEK 2	
-----		-----	
MON	Client submits: "I need customers to upload profile photos"	MON	Sean reviews PRs, merges approved ones, deploys to staging
v		v	
	Sean translates: -> Ticket: "Create image upload API endpoint" -> Ticket: "Build photo upload UI component" -> Ticket: "Configure S3 bucket for uploads"	TUE	Client gets email: "New feature ready for QA testing at qa.clientname.io"
TUE	Alpha starts API endpoint Beta starts UI component Gamma sets up S3 bucket	WED	Client QAs, finds minor issues, submits 2 bug reports
		v	
WED	Alpha opens PR for API Beta opens PR for UI Gamma PR for infra config	THU	Corresponding agent fixes bugs, opens PRs
		v	
THU	Agents iterate on reviews	FRI	Sean approves fixes, deploys to staging again
FRI	Sean does first-pass review, leaves comments	v	
		WEEK 3	

		MON	Client approves staging

```
-> Sean deploys to PROD

TUE Weekly report shows:
[OK] Photo upload shipped
[~] Next: user profiles
```

7.1 Typical Velocity Per Month

Tier	Features/Month	Bug Fixes/Month	Deployments/Month
Solo Agent	2-4 features	5-8 fixes	4-8 deploys
Small Team (3)	5-10 features	10-15 fixes	8-15 deploys
Growth Team (5)	10-20 features	15-25 fixes	12-25 deploys

8. Competitive Advantages & Moat

Moat Element	Why It Works
Human architecture, AI execution	Anyone can run an AI agent. Few can architect a system AND review AI output effectively. You're a senior engineer -- that
Client never touches agents	This prevents competitors from positioning as "rent our AI" and positions you as "we build your software." Different cat
Code ownership for client	No lock-in. Client can leave anytime with full source. Paradoxically, this builds trust that KEEPS them subscribed.
Model improvements = margin growth	As LLMs get better, your agent output quality rises while API costs fall. The business gets BETTER with time, not commod
Per-client isolation	Each client has dedicated agents and servers. No cross-contamination. Security and privacy by design.
Cumulative skill library	With each client project, you build templates, patterns, and agent skills that make the next client FASTER to onboard.
The Sean brand	You're not a faceless agency. Clients hire Sean, not "VETaaS Inc." Personal relationship = retention.

9. Go-to-Market Strategy

9.1 Phase 1: Validation (Months 1-2)

- 2-3 beta clients at 50% discount (\$1,000/mo for Small Team)
- Target: small business owners Sean already knows or can reach
- Goal: validate the workflow, portal, and client satisfaction
- Deliverable: 2-3 case studies + testimonials

9.2 Phase 2: Early Growth (Months 3-6)

- Raise price to \$2,000/mo for new clients
- Target channels:
 - Local business networks -- chambers of commerce, small business meetups
 - Industry-specific -- real estate, healthcare, logistics (sectors with money + software needs)
- Content marketing -- "How we built X for \$2K/month" case study blog posts
- Referral program -- 1 month free for successful referrals
- Goal: 5-8 paying clients

9.3 Phase 3: Scale (Months 7-12)

- Hire 1-2 junior engineers to handle review overflow (trained in your process)
- Standardize tech stack (React/Next.js + Node/Python backend + Railway deploys)
- Build self-serve onboarding: client fills form -> Sean does 1-hour architecture call -> agents start in 48 hours
- Goal: 10-20 clients

9.4 Marketing Angles

Angle	Message
Cost	"A full dev team for the price of a part-time intern"
Speed	"Software shipping every week, not every quarter"
Simplicity	"You describe what you want. We build it. You test it. That's it."
Safety	"You own the code. Cancel anytime. No vendor lock-in."
Quality	"Every line reviewed by a senior engineer before you see it."

10. Financial Projections

10.1 Monthly Runway (Conservative)

Month	Clients	Revenue	Costs	Gross Profit	Sean's Hours
1	2 (beta, \$1K)	\$2,000	\$600	\$1,400	~15h
2	3 (beta, \$1K)	\$3,000	\$900	\$2,100	~20h
3	4 (\$2K retail)	\$8,000	\$1,200	\$6,800	~25h
4	5	\$10,000	\$1,500	\$8,500	~30h
5	7	\$14,000	\$2,100	\$11,900	~35h
6	8	\$16,000	\$2,400	\$13,600	~40h
9	12	\$24,000	\$3,600	\$20,400	~50h
12	15	\$30,000	\$4,500	\$25,500	~55h

At month 12 with a junior engineer hired at \$6K/month to handle review overflow, net would be ~\$19,500/month with Sean's hours dropping to ~25h.

10.2 Annual at Steady State (Year 2)

- 15 clients x \$2,000/mo = \$360,000 annual revenue
- Costs: ~\$54,000 (servers + API) + \$72,000 (1 junior engineer) = \$126,000
- Annual net: ~\$234,000
- Sean's time: ~25 hours/week (sustainable)

11. Risks and Mitigations

Risk	Impact	Mitigation
Agent hallucinates bad code	High -- could ship bugs to production	Sean reviews EVERY PR before merge. Staging QA by client before
Client scope creep	Medium -- endless "one more thing" requests	Prioritized kanban. New requests go to backlog. Client sees queue
API cost spikes	Low-Medium	Monitor usage per client. Consider dedicated models (self-hosted)
Model provider outage	Medium	Maintain accounts with 2+ providers (DeepSeek + Anthropic fallback)
Client churn	Medium -- revenue loss	Code ownership = they can leave, but rebuilding institutional know
Competition from AI agencies	Medium	Human architecture + review layer is the moat. "AI agencies" witho
Sean burnout	High -- single point of failure	Hire junior help at 10+ clients. Document processes. Template eve
Security / data breach	High -- trust destruction	Per-client isolated VPS. No shared databases. Secrets managemen

12. What "Done" Looks Like -- The 12-Month Vision

+-----+

| Sean wakes up Monday. Opens dashboard: |

| |

```

+-----+
| VETaaS -- Operations Dashboard |
|
| 15 Active Clients    $30K MRR    ^ 12% this month |
|
| +-----+ +-----+ +-----+ +-----+ |
| | 12 PRs | | 3 PRs | | 0 Alerts | | All servers | |
| | ready for| | need | | | | [P2] healthy | |
| | review | | revision | | | | | |
| | +-----+ +-----+ +-----+ +-----+ |
| +-----+
|
Sean reviews 12 PRs (~2 hours). Approves 11. Leaves notes
on 1. Agents pick up feedback. Junior engineer handles
client communication for 3 accounts.

Tuesday: Client calls (2 scheduled this week). Architecture
session with a new prospect.

Friday: Weekly reports auto-generate. Sean skims and sends.

Revenue: $30K/month. Sean works ~25 hours/week.
Agents do the typing. Sean does the thinking.
Clients are happy. Software ships.
+-----+

```

13. Next Steps -- What We Need to Build

Priority	Item	Effort	Description
[P0] P0	**Demo client setup**	2-3 days	Spin up 3-agent team on a real project, end-to-end.
[P0] P0	**Agent skill templates**	1-2 days	Create Hermes skill files for Backend, Frontend, and Tooling.
[P1] P1	**Client portal MVP**	1-2 weeks	Simple web dashboard: kanban board (read-only from Hermes), agent status, and chat logs.
[P1] P1	**Weekly report automation**	2-3 days	Cron job that pulls git history, agent activity, and generates a report.
[P1] P1	**Per-client provisioning script**	1 day	Script to spin up 3 Hermes instances + git repo + dependencies.
[P2] P2	**Billing/stripe integration**	1 week	Monthly subscription billing, invoices, client management.
[P2] P2	**Marketing site**	1 week	Landing page, case studies, pricing, contact form.

Appendix A: Sample Client Onboarding Flow

```
Day 0: Client signs up, pays first month
Day 1: Sean has 1-hour architecture call with client
      -> Understands the use case, tech requirements, scope
      -> Sean documents architecture decisions
Day 2: Provisioning script runs:
      -> 3 Hermes agent VPS instances spun up
      -> Private GitHub repo created
      -> Deploy environments configured
      -> Client portal access granted
Day 3: Sean creates first 10-15 tickets (prioritized backlog)
      -> Agents begin work
Day 7: First weekly report sent
      -> Shows initial setup, architecture docs, first tickets in progress
Day 14: First features deployed to staging
      -> Client accesses QA test server
      -> Client submits first round of feedback
Week 3: Feedback incorporated, features refined
Week 4: First production deployment
      -> Full monthly report
      -> Client sees working software for the first time
```

Appendix B: The "Why This Works" -- Sean's Unfair Advantage

You're not selling AI. You're not selling "prompt engineering." You're selling:

"I am a senior software engineer who uses AI as force multipliers. You get the output of a dev team at intern prices, because I know what good looks like and the agents do the typing."

That's a story any small business owner understands. They've been burned by freelancers who disappear. They can't afford agencies. They've tried no-code tools that hit walls.

You're offering: a real dev team, real deployed software, real weekly progress, at a price they can actually afford. The AI is the engine. The trust is the product.

End of Business Plan v1.0

Prepared by Orion (Hermes Agent) for Sean Rohde

Next: Discussion, refinement, or Phase 1 build-out